

南開大學
Nankai University

计算机学院

《人工智能导论》课程实验报告

黑白棋 (Mini AlphaGo)

姓名：赵宇哲

学号：2411094

专业：计算机科学与技术

实验时间：2025 年 10 月 27 日 星期一

目录

1 问题重述 **2**

2 设计思想 **3**

2.1 选择 3

2.2 扩展 3

2.3 模拟 4

2.4 反向传播 4

3 程序代码 **5**

4 测试结果 **9**

5 总结 **11**

5.1 实现 MCTS 四阶段 11

5.2 启发式策略 11

1 问题重述

黑白棋问题

黑白棋 (Reversi)，也叫苹果棋，翻转棋，是一个经典的策略性游戏。

一般棋子双面为黑白两色，故称“黑白棋”。因为行棋之时将对方棋子翻转，则变为己方棋子，故又称“翻转棋”(Reversi)。

棋子双面为红、绿色的称为“苹果棋”。它使用 8x8 的棋盘，由两人执黑子和白子轮流下棋，最后子多方为胜方。

游戏规则：

1. 黑方先行，双方交替下棋。
2. 一步合法的棋步包括：
   (a) 在一个空格处落下一个棋子，并且翻转对手一个或多个棋子；
   (b) 新落下的棋子必须落在可夹住对方棋子的位置上，对方被夹住的所有棋子都要翻转过来，可以是横着夹，竖着夹，或是斜着夹。夹住的位置上必须全部是对手的棋子，不能有空格；
   (c) 一步棋可以在数个（横向，纵向，对角线）方向上翻棋，任何被夹住的棋子都必须被翻转过来，棋手无权选择不去翻某个棋子。
3. 如果一方没有合法棋步，也就是说不管他下到哪里，都不能至少翻转对手的一个棋子，那他这一轮只能弃权，而由他的对手继续落子直到他有合法棋步可下。
4. 如果一方至少有一步合法棋步可下，他就必须落子，不得弃权。
5. 棋局持续下去，直到棋盘填满或者双方都无合法棋步可下。
6. 如果某一方落子时间超过 1 分钟或者连续落子 3 次不合法，则判该方失败。

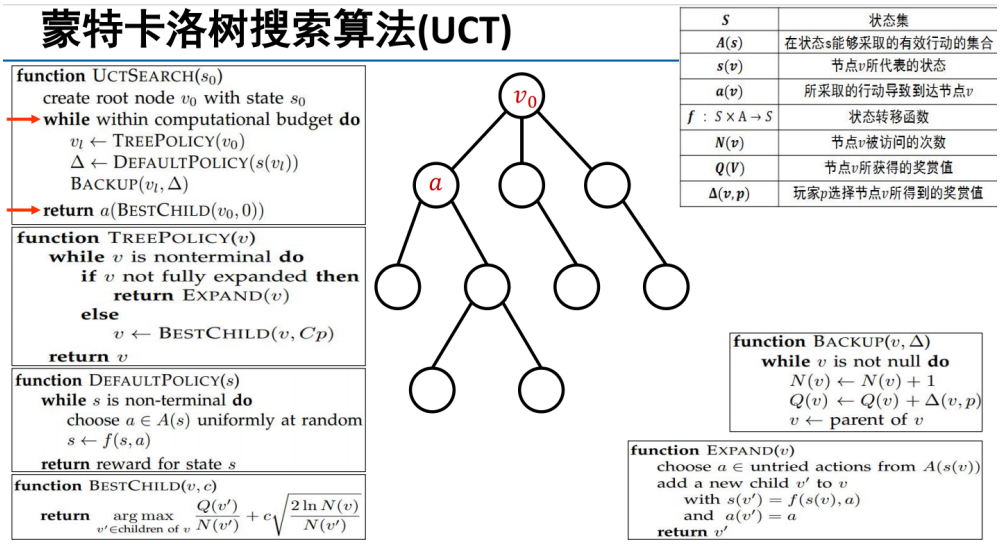
实验要求：

- 使用『蒙特卡洛树搜索算法』实现 miniAlphaGo for Reversi。
- 使用 Python 语言。
- 算法部分需要自己实现，不要使用现成的包、工具或者接口。

问题解读

在 main.py 中实现 AIpayer。

2 设计思想



2.1 选择

有未访问子节点 → 先随机挑一个（鼓励探索）。
否则按带先验的 UCT 得分最大化。

```python
def _select(self, node: MCTSNode) -> MCTSNode:
    unvisited = [c for c in node.children if c.visits == 0]
    if unvisited:
        return random.choice(unvisited)

    ln_parent = math.log(node.visits) if node.visits > 0 else 0.0

    def score(child: MCTSNode):
        exploit = child.wins / child.visits
        explore = self.c_uct * math.sqrt(ln_parent / child.visits)
        bias    = self.bias_k * (child.prior / (1.0 + child.visits))
        return exploit + explore + bias

    return max(node.children, key=score)
```

2.2 扩展

当前应走方 to_move = opponent(node.player_just_moved)。
有合法步：为每步建子。无合法步：若对手也无步 → 终局；否则建 pass 子。

```python
def _expand(self, node: MCTSNode, board) -> None:
    to_move = _opponent(node.player_just_moved)
    legal = list(board.get_legal_actions(to_move))

    if legal:
```

```
        for mv in legal:
            node.add_child(mv, to_move)
    else:
        if not list(board.get_legal_actions(_opponent(to_move))):
            return  # 终局，不扩展
        node.add_child(None, to_move)  # pass
```

2.3 模拟

一方无步 →pass；双方都无步 → 终局。

返回 winner_code（0:X 胜，1:O 胜，2: 平）。

```
def _simulate(self, board, color_start):
    cur = color_start
    while True:
        legal = list(board.get_legal_actions(cur))
        if not legal:
            other = _opponent(cur)
            if not list(board.get_legal_actions(other)):
                break  # 双无步，终局
            cur = other  # pass
            continue

        mv = self._policy_move(legal)
        board._move(mv, cur)
        cur = _opponent(cur)

    winner_code, _ = board.get_winner()
    return winner_code
```

2.4 反向传播

节点 visits += 1；

胜者等于 player_just_moved → wins += 1；平局 → wins += 0.5；否则加 0。

因为视角一致，父子统计的一致性很好，避免正负号混乱。

```
def _backpropagate(self, node: MCTSNode, winner_code: int) -> None:
    cur = node
    while cur is not None:
        cur.visits += 1
        if winner_code == 2:
            reward = 0.5
        else:
            winner_color = "X" if winner_code == 0 else "O"
            reward = 1.0 if cur.player_just_moved == winner_color else 0.0
        cur.wins += reward
        cur = cur.parent
```

3 程序代码

```python
import random
import math
from copy import deepcopy


# ———————— 小工具：对坐标做轻量启发 ————————
_COLS = "ABCDEFGH"
_CORNERS = {"A1", "A8", "H1", "H8"}
_X_SQUARES = {"B2", "B7", "G2", "G7"}           # 角的对角邻点
_C_SQUARES = {"A2", "B1", "A7", "B8", "H2", "G1", "H7", "G8"}  # 角的正邻点


def _is_edge(m):
    return (m[0] in ("A", "H")) or (m[1] in ("1", "8"))


def _prior_of(move):
    """落子先验分（用于 Progressive Bias；数值只相对有意义）"""
    if move is None:
        return -5.0
    m = move.upper()
    if m in _CORNERS:
        return 100.0
    if m in _X_SQUARES:
        return -40.0
    if m in _C_SQUARES:
        return -20.0
    if _is_edge(m):
        return 8.0
    return 0.0


def _opponent(c):
    return "O" if c == "X" else "X"


# ———————— MCTS 节点 ————————
class MCTSNode:
    __slots__ = ("parent", "children", "move", "wins", "visits",
                 "player_just_moved", "prior")

    def __init__(self, parent=None, move=None, player_just_moved=None, prior=0.0):
        self.parent = parent
        self.children = []
        self.move = move                        # 导致该节点的着法（根为 None；pass 也用 None）
```

```
        self.wins = 0.0                          # 以 player_just_moved 视角计算
        self.visits = 0
        self.player_just_moved = player_just_moved
        self.prior = prior                       # Progressive Bias 的先验

    def add_child(self, move, player_just_moved):
        child = MCTSNode(self, move, player_just_moved, _prior_of(move))
        self.children.append(child)
        return child

    def is_leaf(self):
        return len(self.children) == 0


# ———————— 主体 AI ————————
class AIPlayer:
    def __init__(self, color: str):
        self.color = color.upper()
        self.comments = "请稍后, {}正在思考".format("黑棋(X)" if self.color == "X"
            else "白棋(O)")
        # 可按需微调
        self.iterations = 3000              # 迭代次数
        self.c_uct = math.sqrt(2.0)         # UCT 探索系数
        self.bias_k = 0.25                  # Progressive Bias 系数（0 关闭）
        self.rollout_epsilon = 0.10         # 重放中以   概率随机选子，避免过度贪心

    # ———————— 选择 ————————
    def _select(self, node: MCTSNode) -> MCTSNode:
        # 有未访问子节点：优先随即挑一个（鼓励探索）
        unvisited = [c for c in node.children if c.visits == 0]
        if unvisited:
            return random.choice(unvisited)

        ln_parent = math.log(node.visits) if node.visits > 0 else 0.0

        def score(child: MCTSNode):
            exploit = child.wins / child.visits
            explore = self.c_uct * math.sqrt(ln_parent / child.visits)
            bias = self.bias_k * (child.prior / (1.0 + child.visits))
            return exploit + explore + bias

        return max(node.children, key=score)

    # ———————— 扩展 ————————
    def _expand(self, node: MCTSNode, board) -> None:
        to_move = _opponent(node.player_just_moved)
        legal = list(board.get_legal_actions(to_move))

        if legal:
```

```
            for mv in legal:
                node.add_child(mv, to_move)
        else:
            # 当前无棋可下，若对手也无则终局；否则生成 pass 子节点
            if not list(board.get_legal_actions(_opponent(to_move))):
                return
            node.add_child(None, to_move)

    # ———————— 重放策略 ————————
    def _policy_move(self, legal):
        """
        轻量启发式：角 > (排除 X) > 边 > 其他；加 -随机。
        """
        legal = [m.upper() for m in legal]
        if random.random() < self.rollout_epsilon:
            return random.choice(legal)

        corners = [m for m in legal if m in _CORNERS]
        if corners:
            return random.choice(corners)

        good = [m for m in legal if (m not in _X_SQUARES and m not in _C_SQUARES)]
        edges = [m for m in good if _is_edge(m)]
        if edges:
            return random.choice(edges)

        # 没法避免，只能在余下集合里随机
        return random.choice(good if good else legal)

    # ———————— 模拟 ————————
    def _simulate(self, board, color_start):
        cur = color_start
        while True:
            legal = list(board.get_legal_actions(cur))
            if not legal:
                other = _opponent(cur)
                if not list(board.get_legal_actions(other)):
                    break  # 双方都无步，终局
                # 当前方 pass
                cur = other
                continue

            mv = self._policy_move(legal)
            board._move(mv, cur)
            cur = _opponent(cur)

        winner_code, _diff = board.get_winner()  # 0->X胜，1->O胜，2->平
        return winner_code
```

```
    # ———————— 回传 ————————
    def _backpropagate(self, node: MCTSNode, winner_code: int) -> None:
        cur = node
        while cur is not None:
            cur.visits += 1
            if winner_code == 2:
                reward = 0.5
            else:
                winner_color = "X" if winner_code == 0 else "O"
                reward = 1.0 if cur.player_just_moved == winner_color else 0.0
            cur.wins += reward
            cur = cur.parent

    # ———————— 主流程 ————————
    def get_move(self, board):
        print(self.comments)

        # 根：认为“上一手是对方刚走完”，便于用 player_just_moved 视角
        root = MCTSNode(parent=None, move=None,
            player_just_moved=_opponent(self.color), prior=0.0)

        for _ in range(self.iterations):
            node = root
            sim_board = deepcopy(board)

            # 1) 选择：沿树下探到叶子（一路在副本棋盘上执行动作）
            while not node.is_leaf():
                child = self._select(node)
                # 应用被选子节点的着法（pass=None 时不落子）
                to_move = _opponent(node.player_just_moved)
                if child.move is not None:
                    sim_board._move(child.move, to_move)
                node = child

            # 2) 扩展：非终局才扩展
            to_move = _opponent(node.player_just_moved)
            if list(sim_board.get_legal_actions(to_move)) or list(
                    sim_board.get_legal_actions(_opponent(to_move))):
                self._expand(node, sim_board)

            # 3) 模拟：从一个未访问子节点（若存在）或 UCT 子节点开始 rollout
            rollout_from = node
            if node.children:
                unvisited = [c for c in node.children if c.visits == 0]
                rollout_from = random.choice(unvisited) if unvisited else
                    self._select(node)
                # 应用 rollout 起点那一步
                to_move = _opponent(node.player_just_moved)
                if rollout_from.move is not None:
```

```
                sim_board._move(rollout_from.move, to_move)
            start_color = _opponent(to_move)
        else:
            start_color = to_move  # 叶即终局或无子可扩展

        winner_code = self._simulate(sim_board, start_color)

        # 4) 回传
        self._backpropagate(rollout_from, winner_code)

    # 没子说明无合法步（极少见，game 控制流一般不会调用到这里）
    if not root.children:
        return None

    # 访问次数最多的子节点（也可换成胜率最高）
    best = max(root.children, key=lambda c: c.visits)
    return best.move
```

4 测试结果

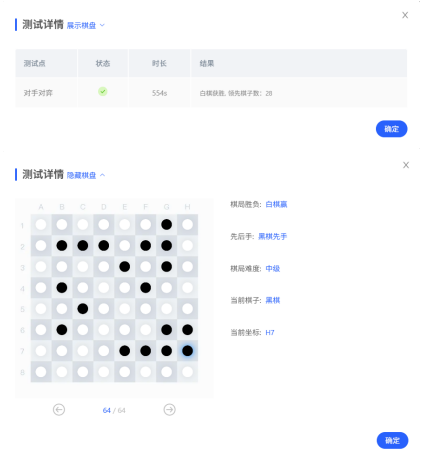
测试点 1：初级难度 黑棋先手



测试点 2：初级难度 白棋后手



测试点 3：中级难度 黑棋先手



测试点 4：中级难度 白棋后手



测试点 5：高级难度 黑棋先手



测试点 6：高级难度 白棋后手



5 总结

5.1 实现 MCTS 四阶段

- 在 `get_move` 中以循环完成以下步骤：
  - 选择（Selection）
  - 扩展（Expansion）
  - 模拟（Simulation）
  - 回传（Backpropagation）
- 在选择路径上真实落子到 `sim_board=deepcopy(board)`，保证树状态与模拟状态一致（`_select` + 路径 `_move`）。
- 节点统计以 `player_just_moved` 视角计分，回传时胜 =1、和 =0.5、负 =0（`_backpropagate`），使父子节点的胜率含义统一。

5.2 启发式策略

- 渐进偏置：在 UCT 里加入 `bias_k * prior/(1+visits)`（`_select` 与 `_prior_of`），用 “角 > 边 > 内、惩罚 X/C 点” 的先验引导前期搜索，访问增多后自动衰减。
- 重放策略：Rollout 不再纯随机（`_policy_move`），按 “角优先 → 避 X/C→ 边 → 其余” 的顺序，并加 `rollout_epsilon` 的少量随机，显著降低模拟方差。